Heidelberg University
Hardware and Artificial Intelligence (HAWAII) Lab

Holger Fröning
Arjan Siddhpura and Gina Rachor

High-Performance and Distributed Computing
Summer 2025

Exercise 5

- **Hand in via Moodle until 09:00 on Thursday 25 June, 2026**
- **Include all names on the top sheet. Hand in a single PDF.**
- **Compress your results into a single archive (`.zip` or `.tar.gz`).**
- **Employ the following naming convention `hpc<XX>_ex<N>.{zip,tar.gz}`, where `XX` is your group number, and `N` is the number of the current exercise.**
- **A maximum of three students are allowed to collaborate on the exercises.**
- **In case an exercise requires programming:**
  - **include clean and documented code**
  - **include a Makefile for compiling**

5.1 Implementation of Ring Allreduce

- The final goal is to implement the Ring Allreduce as discussed in the lecture. It is composed out of a Ring Reduce-Scatter, followed by a Ring Allgather.
- Before the algorithm is run, all processes have contiguous arrays of floats with the same size. In the end, for each array index, all processes should have the sum of the corresponding values in the same place.
- For the first step, the arrays have to be split into chunks (continuous subarrays). In order for the algorithm to run fast, their size should be as equal as possible.
- Think of the size of the chunks in a ring structure, i.e. the successor of the last chunk is the first one and the predecessor of the first one is the last one. There are as many chunks as processes (size s). When implementing this behavior, keep in mind that % calculates the remainder but not the modulo operator in C.
- In the i-th iteration of Ring Reduce-Scatter, the process with rank r sends chunk $(r-i) \mod s$ to the next process, receives chunk $(r-(i+1)) \mod s$ from the previous process and adds the float it receives to the corresponding ones in his current array. At the end of this part, the floats in the $(r+1) \mod s$-th chunk of the process with rank r are the sum of all floats from this chunk from all processes. The other chunks in each process contain partial sums that will be overridden in the next step.
- In the i-th iteration of the Ring Allgather, the process with rank r sends chunk $(r+1-i) \mod s$ to its successor and receives chunk $(r-i) \mod s$ to its predecessor. It overrides its current chunk with what it receives.
- When you check your solution for correctness, for example by comparing with the MPI native solution, mind to tolerate small (epsilon) deviations. In particular, if you use larger floats as input, use a relative epsilon (i.e. tolerate deviations by e.g. 0,001 %). Floating point inaccuracies are integer values if the numbers get higher!

(50 points)

5.2 Ring Allreduce – scaling problem size

- Execute your solution (Ring Allreduce) and MPI's native one on octane nodes with 8 processes. Vary the problem size in powers of two from 2^{16} to 2^{22} floats. Measure the execution time (excluding time for initialization etc.). Average the runtime over five runs. Compare with MPI natives solution.
- Use two contrary distribution of processes on nodes: Make one experiment with the sbatch options `"--nodes=1"` `"-tasks-per-node=8"` so that all processes run on one node and another experiment with `"--nodes=8"` and `"--tasks-per-node=1"` so that each process is on a different node. In the latter case, contrary to all other exercises, **do NOT use** `"--exclusive"` as you than would need the entire octane cluster which is impossible as there are always some jobs running from the research of the HAWAII group. Use squeue to make sure you don't interfere with other exercise groups, interference with other jobs on the octane cluster is not so important.
- You should beat the native solution in the latter experiment!

Problem Size	Ring Allreduce, One Node [s]	MPI Native, One Node [s]	Ring Allreduce, One Node per Process [s]	MPI Native, One Node per Process [s]
2^{16}				
2^{17}				
2^{18}				
...				
2^{22}				

(25 points)

5.3 Ring Allreduce - scaling process count

- Execute on the HPDC exercise partition (octane nodes) and report execution times and speed-up for 2, 3, 4, ..., 8 processes (increment of 1) operating on 1'000'000 floats. Measure the execution time (excluding time for initialization). Average the runtime over five runs. Compare with the native MPI solution.
- Use the same two different sbatch configurations as in 5.2. Again, you will beat the native solution if all processes reside on different nodes!

Process Count	Ring Allreduce, One Node [s]	MPI Native, One Node [s]	Ring Allreduce, One Node per Process [s]	MPI Native, One Node per Process [s]
2				
3				
4				
...				
8				

Table 1: Execution tiem t_{ex} in $[s]$

Process Count	Ring Allreduce, One Node	MPI Native, One Node	Ring Allreduce, One Node per Process	MPI Native, One Node per Process
2				
3				
4				
...				
8				

Table 2: Speedup a

(25 points)
Notes

- Ensure that the compute nodes are free when performing experiments. In particular, look for other participants which could be performing tests. Try not to disturb them; you also might not want to be disturbed.
- Add plots where helpful for improved interpretation! Often trends are visible more clearly when data is reported graphically.
- Remeber to analyze and interpret your results.

5.4 Willingness to present

Please declare whether you are willing to present any of the previous exercises.
The declaration can be made on a per-exercise basis. Each declaration corresponds to 50% of the exercise points. You can only declare your willingness to present exercises for which you also hand in a solution. If no willingness to present is declared you may still be required to present an exercise for which your group has handed in a solution. This may happen if as example nobody else has declared their willingness to present.

- Ring Allreduce - Implementation (Section: 5.1)
- Ring Allreduce - Scaling problem size (Section: 5.2)
- Ring Allreduce - Sacling process count (Section: 5.3)

(25+12.5+12.5 points)

Total: 100+50 points